

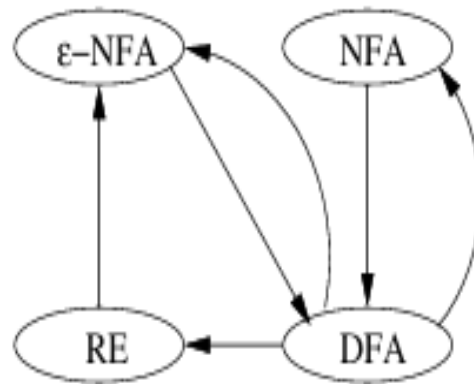
CSE 3204
Formal Language, Automata and Computability



Lecture 6
Finite Automata and Regular Expression

Regular Expression and Finite Automata

- Regular expression approach and finite automata approach represent same set of languages known as Regular Language.



- Language accepted by some DFA is also defined by regular expression
- Language defined by a regular expression is also accepted by a NFA with ϵ -transition.

From DFA's to Regular Expression

- Construct expressions that describe sets of strings that label certain paths in the DFA. The paths are allowed to pass through only a limited subset of the states.
- Start with the simplest expressions that describe paths that are not allowed to pass through any states (single nodes or single arcs)
- Inductively build the expressions that let the paths go through progressively larger sets of states.

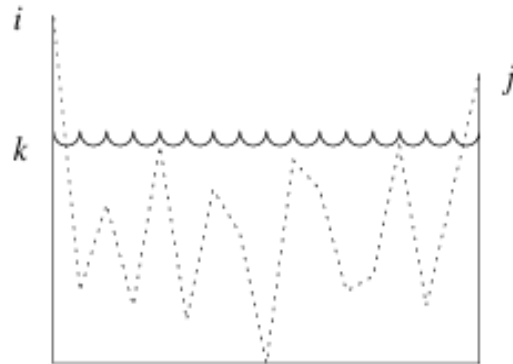
From DFA's to Regular Expression

Theorem 3.4: If $L = L(A)$ for some DFA A , then there is a regular expression R such that $L = L(R)$.

Let us use $R_{ij}^{(k)}$ as the name of a regular expression whose language is the set of strings w such that w is the label of a path from state i to state j in A , and that path has no intermediate node whose number is greater than k . Note that the beginning and end points of the path are not “intermediate,” so there is no constraint that i and/or j be less than or equal to k .

BASIS: The basis is $k = 0$. Since all states are numbered 1 or above, the restriction on paths is that the path must have no intermediate states at all. There are only two kinds of paths that meet such a condition:

1. An arc from node (state) i to node j .
2. A path of length 0 that consists of only some node i .



If $i \neq j$, then only case (1) is possible. We must examine the DFA A and find those input symbols a such that there is a transition from state i to state j on symbol a .

- a) If there is no such symbol a , then $R_{ij}^{(0)} = \emptyset$.
- b) If there is exactly one such symbol a , then $R_{ij}^{(0)} = \mathbf{a}$.
- c) If there are symbols $a_1, a_2, \dots, a_k$ that label arcs from state i to state j , then $R_{ij}^{(0)} = \mathbf{a_1 + a_2 + \dots + a_k}$.

From DFA's to Regular Expression

However, if $i = j$, then the legal paths are the path of length 0 and all loops from i to itself. The path of length 0 is represented by the regular expression ϵ , since that path has no symbols along it. Thus, we add ϵ to the various expressions devised in (a) through (c) above. That is, in case (a) [no symbol a] the expression becomes ϵ , in case (b) [one symbol a] the expression becomes $\epsilon + \mathbf{a}$, and in case (c) [multiple symbols] the expression becomes $\epsilon + \mathbf{a}_1 + \mathbf{a}_2 + \cdots + \mathbf{a}_k$.

INDUCTION: Suppose there is a path from state i to state j that goes through no state higher than k . There are two possible cases to consider:

1. The path does not go through state k at all. In this case, the label of the path is in the language of $R_{ij}^{(k-1)}$.
2. The path goes through state k at least once. Then we can break the path into several pieces, as suggested by Fig. 3.3. The first goes from state i to state k without passing through k , the last piece goes from k to j without passing through k , and all the pieces in the middle go from k to itself, without passing through k . Note that if the path goes through state k only once, then there are no “middle” pieces, just a path from i to k and a path from k to j . The set of labels for all paths of this type is represented by the regular expression $R_{ik}^{(k-1)} (R_{kk}^{(k-1)})^* R_{kj}^{(k-1)}$. That is,

From DFA's to Regular Expression



Figure 3.3: A path from i to j can be broken into segments at each point where it goes through state k

When we combine the expressions for the paths of the two types above, we have the expression

$$R_{ij}^{(k)} = R_{ij}^{(k-1)} + R_{ik}^{(k-1)} (R_{kk}^{(k-1)})^* R_{kj}^{(k-1)}$$

From DFA's to Regular Expression-Example

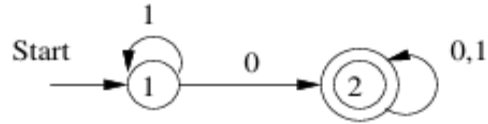


Figure 3.4: A DFA accepting all strings that have at least one 0

Basis Expression:

$R_{11}^{(0)}$	$\epsilon + 1$
$R_{12}^{(0)}$	0
$R_{21}^{(0)}$	$\emptyset$
$R_{22}^{(0)}$	$(\epsilon + 0 + 1)$

$$R_{ij}^{(1)} = R_{ij}^{(0)} + R_{i1}^{(0)}(R_{11}^{(0)})^*R_{1j}^{(0)} \quad (3.1)$$

	By direct substitution	Simplified
$R_{11}^{(1)}$	$\epsilon + 1 + (\epsilon + 1)(\epsilon + 1)^*(\epsilon + 1)$	1^*
$R_{12}^{(1)}$	$0 + (\epsilon + 1)(\epsilon + 1)^*0$	1^*0
$R_{21}^{(1)}$	$\emptyset + \emptyset(\epsilon + 1)^*(\epsilon + 1)$	$\emptyset$
$R_{22}^{(1)}$	$\epsilon + 0 + 1 + \emptyset(\epsilon + 1)^*0$	$\epsilon + 0 + 1$

Figure 3.5: Regular expressions for paths that can go through only state 1

Simplification rule:

1. $(\epsilon + R)^* = R^*$
2. $(\epsilon + 1)1^* = 1^*$.
3. $\emptyset R = R\emptyset = \emptyset$. That is, $\emptyset$ is an *annihilator* for concatenation
4. $\emptyset + R = R + \emptyset = R$

From DFA's to Regular Expression-Example

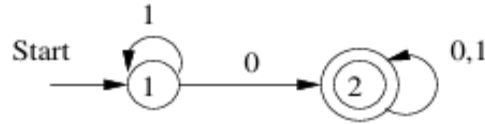


Figure 3.4: A DFA accepting all strings that have at least one 0

Basis Expression:

	By direct substitution	Simplified
$R_{11}^{(1)}$	$\epsilon + 1 + (\epsilon + 1)(\epsilon + 1)^*(\epsilon + 1)$	1^*
$R_{12}^{(1)}$	$0 + (\epsilon + 1)(\epsilon + 1)^*0$	1^*0
$R_{21}^{(1)}$	$\emptyset + \emptyset(\epsilon + 1)^*(\epsilon + 1)$	$\emptyset$
$R_{22}^{(1)}$	$\epsilon + 0 + 1 + \emptyset(\epsilon + 1)^*0$	$\epsilon + 0 + 1$

Figure 3.5: Regular expressions for paths that can go through only state 1

$$R_{ij}^{(2)} = R_{ij}^{(1)} + R_{i2}^{(1)}(R_{22}^{(1)})^*R_{2j}^{(1)}$$

Final Regular expression: $R_{12}^{(2)} = 1^*0(0 + 1)^*$

	By direct substitution	Simplified
$R_{11}^{(2)}$	$1^* + 1^*0(\epsilon + 0 + 1)^*\emptyset$	1^*
$R_{12}^{(2)}$	$1^*0 + 1^*0(\epsilon + 0 + 1)^*(\epsilon + 0 + 1)$	$1^*0(0 + 1)^*$
$R_{21}^{(2)}$	$\emptyset + (\epsilon + 0 + 1)(\epsilon + 0 + 1)^*\emptyset$	$\emptyset$
$R_{22}^{(2)}$	$\epsilon + 0 + 1 + (\epsilon + 0 + 1)(\epsilon + 0 + 1)^*(\epsilon + 0 + 1)$	$(0 + 1)^*$

Figure 3.6: Regular expressions for paths that can go through any state

Simplification rule:

1. $(\epsilon + R)^* = R^*$
2. $(\epsilon + 1)1^* = 1^*$.
3. $\emptyset R = R\emptyset = \emptyset$. That is, $\emptyset$ is an *annihilator* for concatenation
4. $\emptyset + R = R + \emptyset = R$

DFA to Regular Expression by Eliminating States

- Method discussed in earlier slides works on any FA.
- the construction of the regular expression is expensive.
- Need to construct about n^3 expressions for an n state automaton.
- Also the length of the expression can grow by a factor of 4 on the average with each of the n inductive steps.
- Thus the expressions themselves could reach on the order of 4^n symbols
- The mechanism is to eliminate state s in the path from q to p .
- Label of the arc is a regular expression.

DFA to Regular Expression by Eliminating States

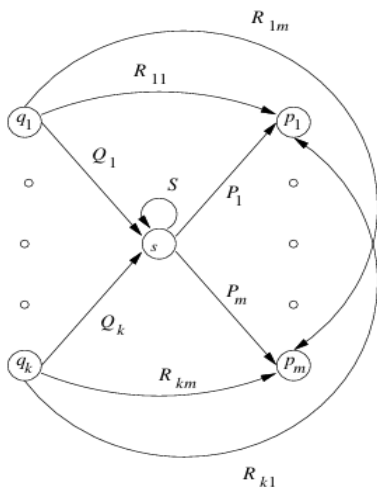


Figure 3.7: A state s about to be eliminated

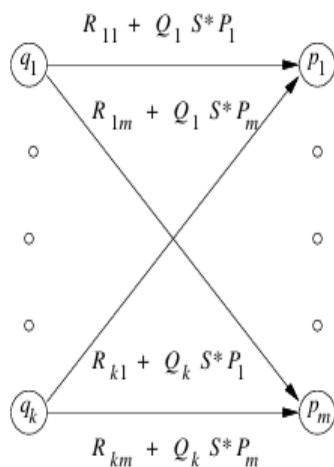


Figure 3.8: Result of eliminating state s from Fig. 3.7

1. For each accepting state q , apply the above reduction process to produce an equivalent automaton with regular-expression labels on the arcs. Eliminate all states except q and the start state q_0 .
2. If $q \neq q_0$, then we shall be left with a two-state automaton that looks like Fig. 3.9. The regular expression for the accepted strings can be described in various ways. One is $(R + SU^*T)^*SU^*$. In explanation, we can go from the start state to itself any number of times, by following a sequence of paths whose labels are in either $L(R)$ or $L(SU^*T)$. The expression SU^*T represents paths that go to the accepting state via a path in $L(S)$, perhaps return to the accepting state several times using a sequence of paths with labels in $L(U)$, and then return to the start state with a path whose label is in $L(T)$. Then we must go to the accepting state, never to return to the start state, by following a path with a label in $L(S)$. Once

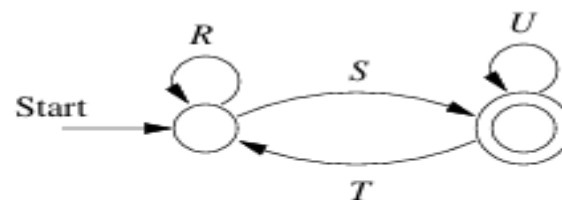


Figure 3.9: A generic two-state automaton

DFA to Regular Expression by Eliminating States

3. If the start state is also an accepting state, then we must also perform a state-elimination from the original automaton that gets rid of every state but the start state. When we do so, we are left with a one-state automaton that looks like Fig. 3.10. The regular expression denoting the strings that it accepts is R^* .

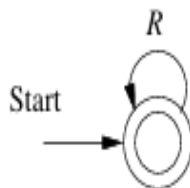


Figure 3.10: A generic one-state automaton

4. The desired regular expression is the sum (union) of all the expressions derived from the reduced automata for each accepting state, by rules (2) and (3).

• Example

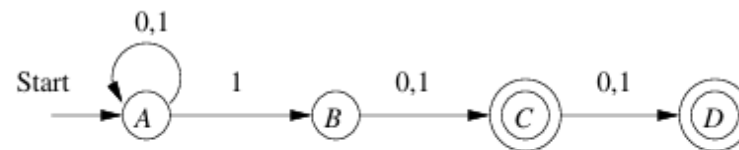


Figure 3.11: An NFA accepting strings that have a 1 either two or three positions from the end

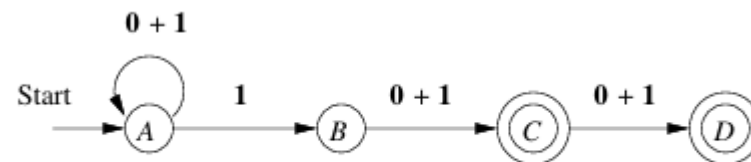


Figure 3.12: The automaton of Fig. 3.11 with regular-expression labels

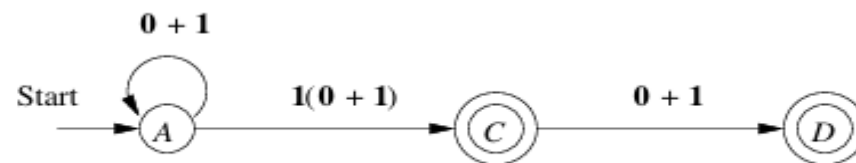


Figure 3.13: Eliminating state B

DFA to Regular Expression by Eliminating States

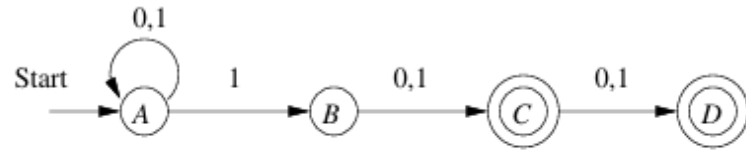


Figure 3.11: An NFA accepting strings that have a 1 either two or three positions from the end

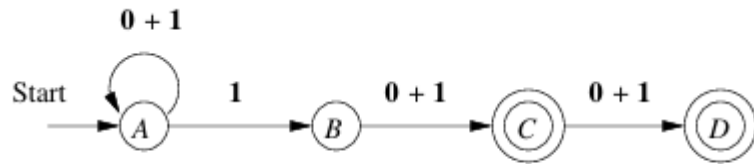


Figure 3.12: The automaton of Fig. 3.11 with regular-expression labels

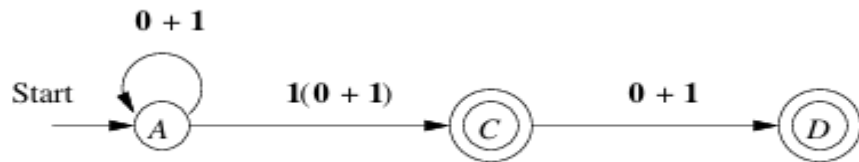


Figure 3.13: Eliminating state B

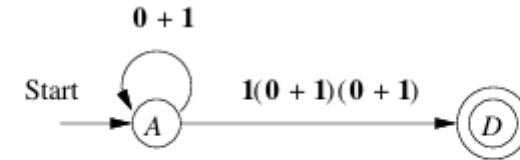


Figure 3.14: A two-state automaton with states A and D

The generic expression $(R + SU^*T)^*SU^*$ thus simplifies in this case to R^*S , or $(0 + 1)^*1(0 + 1)(0 + 1)$. In informal terms, the language of this expression is any string ending in 1, followed by two symbols that are each either 0 or 1. That language is one portion of the strings accepted by the automaton of Fig. 3.11: those strings whose third position from the end has a 1.

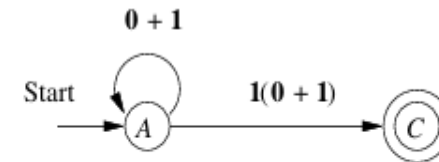


Figure 3.15: Two-state automaton resulting from the elimination of D

String is $(0 + 1)^* \hat{1}(\hat{0} + \hat{1})$.

DFA to Regular Expression by Eliminating States

All that remains is to sum the two expressions to get the expression for the entire automaton of Fig. 3.11. This expression is

$$(0 + 1)^* 1(0 + 1) + (0 + 1)^* 1(0 + 1)(0 + 1)$$

- Section 3.2.1 and 3.2.2 of Chapter 3